

AuditLens — Agentic Invoice & Expense Auditor

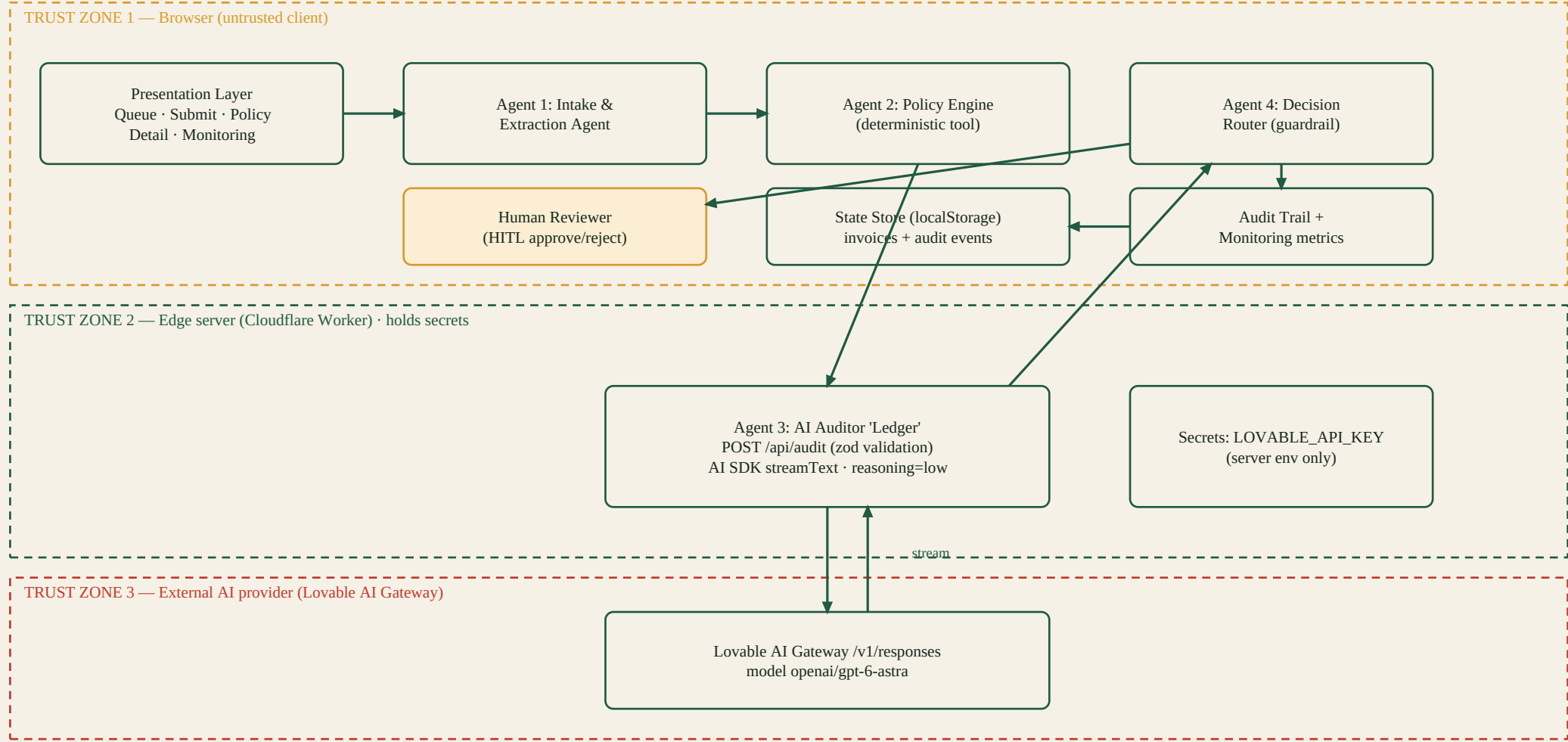
Capstone Deliverables · Enabling Scalable Enterprises using Agentic AI

Student: Jeffin Solomon Asir J S **Roll No:** 2023103501

AuditLens is a multi-agent system that captures invoices and employee expense claims, enforces company policy with a deterministic rules engine, uses an LLM auditor ("Ledger") to reason about anomalies and fraud signals, and routes every claim either to auto-approval or to a human finance reviewer. Contents: 1. Architecture Diagram · 2. Agent Workflow Design · 3. Deployment Strategy · 4. Security Model · 5. Monitoring Dashboard Design.

1. Architecture Diagram

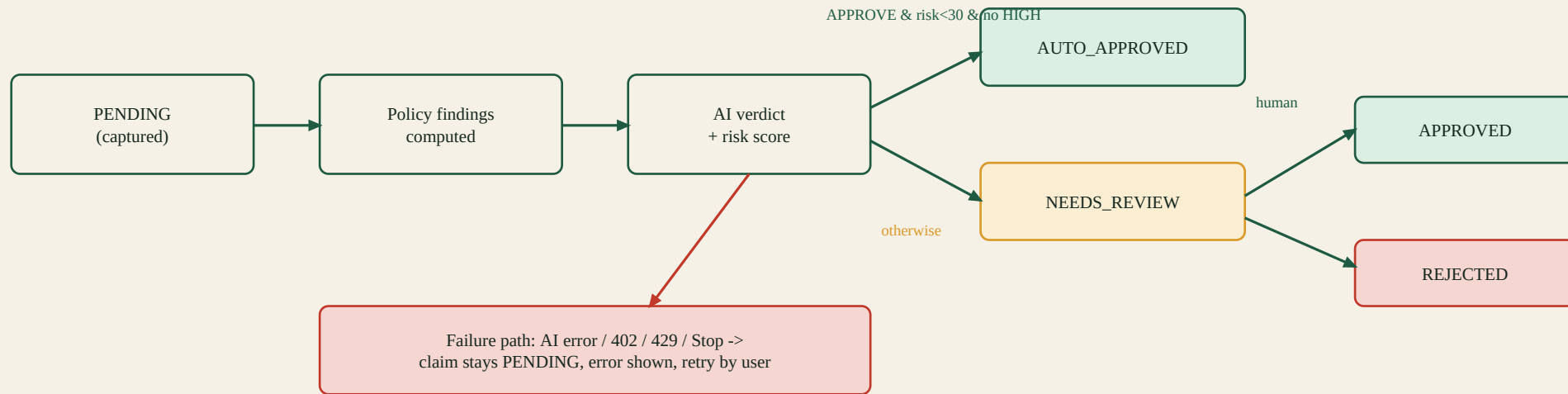
Layers, components, trust boundaries and integrations.



Layer	Components	Integration
Presentation	TanStack Start React routes, Tailwind design system, Recharts	Browser
Agent orchestration	Intake -> Policy Engine -> AI Auditor -> Router, run step-by-step with progress UI	Client orchestrator + server route
Intelligence	Ledger LLM auditor (Responses API, streaming text, VERDICT JSON contract)	Lovable AI Gateway over HTTPS
Observability	Monitoring page, trace log, AI gateway request logs (run-id header)	Gateway logs

2. Agent Workflow Design

Roles, states, tools, handoffs, approvals and failure paths.



Agent / role	Responsibility	Tools / inputs	Hand-off
1. Intake & Extraction	Capture claim, structure line items, totals, GSTIN, receipt flag	Submit form, seed data	Structured Invoice JSON -> Policy Engine
2. Policy Engine	Deterministic checks: receipt, duplicate, >Rs. 50k, GSTIN, 30-day, weekend, alcohol, hotel cap, meal cap	runPolicyEngine(invoice, ledger)	Findings[] (rule, severity, detail) -> AI Auditor
3. AI Auditor 'Ledger'	Contextual reasoning, anomaly/fraud signals, recommendation, risk 0-100	LLM via /api/audit, policy text, findings	Markdown report + VERDICT JSON -> Router
4. Decision Router	Guardrail combining AI verdict with hard rules; AI can never pay or reject alone	routeDecision()	AUTO_APPROVED or NEEDS_REVIEW -> Human
Human Finance Reviewer	Final approve/reject for escalated claims	HITL panel	APPROVED / REJECTED + AuditEvent

Failure paths: invalid request -> 400; missing key -> 500 config error; credits exhausted -> 402 message; rate limited -> 429 message; user Stop -> abort signal propagated to the model call; malformed VERDICT -> defaults to REVIEW/risk 50 (fail-safe to human). No automatic retries, so no duplicate billing.

3. Deployment Strategy

Runtime, scaling, resilience, environments and release.

Aspect	Strategy
Runtime	TanStack Start SSR + server route /api/audit on Cloudflare Workers (edge, V8 isolates) via Lovable hosting; static assets on CDN.
Scaling	Stateless workers scale horizontally per request; AI calls are streamed so long audits do not hold buffers; the gateway shares a workspace rate-limit budget — batch audits sequentially.
Resilience	Streaming with Stop/abort; bounded error handling for 402/429/5xx; fail-safe routing to human review; deterministic policy engine still works if AI is down (findings shown).
Environments	Preview (id-preview URL, latest build) for development/testing; Production (published lovable.app URL). Secrets set per environment, never in code.
Release	Git-synced repo (GitHub two-way sync) -> build checks (TypeScript, Vite) -> preview verification (Playwright E2E audit run) -> Publish. Rollback via version history.
Production hardening (future)	Replace localStorage with Lovable Cloud Postgres + RLS, add SSO login, queue-based batch audits with background jobs, OCR intake for PDF/image invoices.

4. Security Model

Identity, authorization, secrets, privacy, guardrails and audit.

Control	Implementation
Identity	Demo personas (employee submitter, finance reviewer). Production: SSO / OAuth via Lovable Cloud auth.
Authorization	Separation of duties: AI agents can only recommend; only the human reviewer role can APPROVE/REJECT escalated claims. Production: roles in a separate user_roles table checked server-side with RLS.
Secrets	LOVABLE_API_KEY read only inside the server handler (process.env); never shipped to the browser or stored in localStorage.
Privacy	Only the single claim being audited is sent to the model; no PII beyond employee name; requests use store:false (gateway is stateless, no provider-side retention of history).
Input validation	zod schema on /api/audit rejects malformed payloads (400).
Guardrails	Deterministic rules run before the LLM; HIGH findings block auto-approval; system prompt forbids invented facts; strict VERDICT contract; malformed output fails safe to human review; word limit on output.
Prompt-injection defence	Claim fields (notes, descriptions) are passed as JSON data, not instructions; router decisions do not depend solely on model text.
Audit	Append-only AuditEvent log (timestamp, actor, action) for every agent verdict and human decision, shown per claim and on Monitoring; gateway run-ID header links requests to AI logs.

5. Monitoring Dashboard Design

Health, trace, quality, safety, cost and business outcomes.



Dimension	Metric	Source	Alert idea
Health	Avg end-to-end pipeline latency; error rate	latencyMs per audit; failed runs	p95 > 30s or errors > 5%
Trace	Per-claim trace of agent steps and human actions	AuditEvent log + gateway run-ID	Missing VERDICT line
Quality	Claims audited, AI vs human agreement	status vs human decision	Override rate > 20%
Safety	Human overrides, HIGH findings blocked from auto-approval	events, findings	Any auto-approval with HIGH finding (must be 0)
Cost	Estimated AI cost per audit and total	audit count × unit cost; gateway usage	Monthly budget 80%
Business outcome	Auto-approval rate, Rs. leakage prevented, value audited	invoice totals by status	Leakage trend